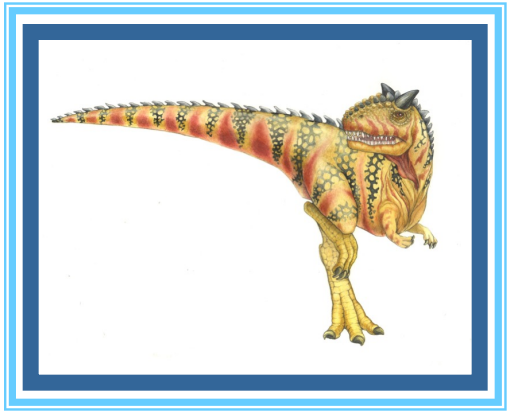
# Chapter 4: Threads



# Topics

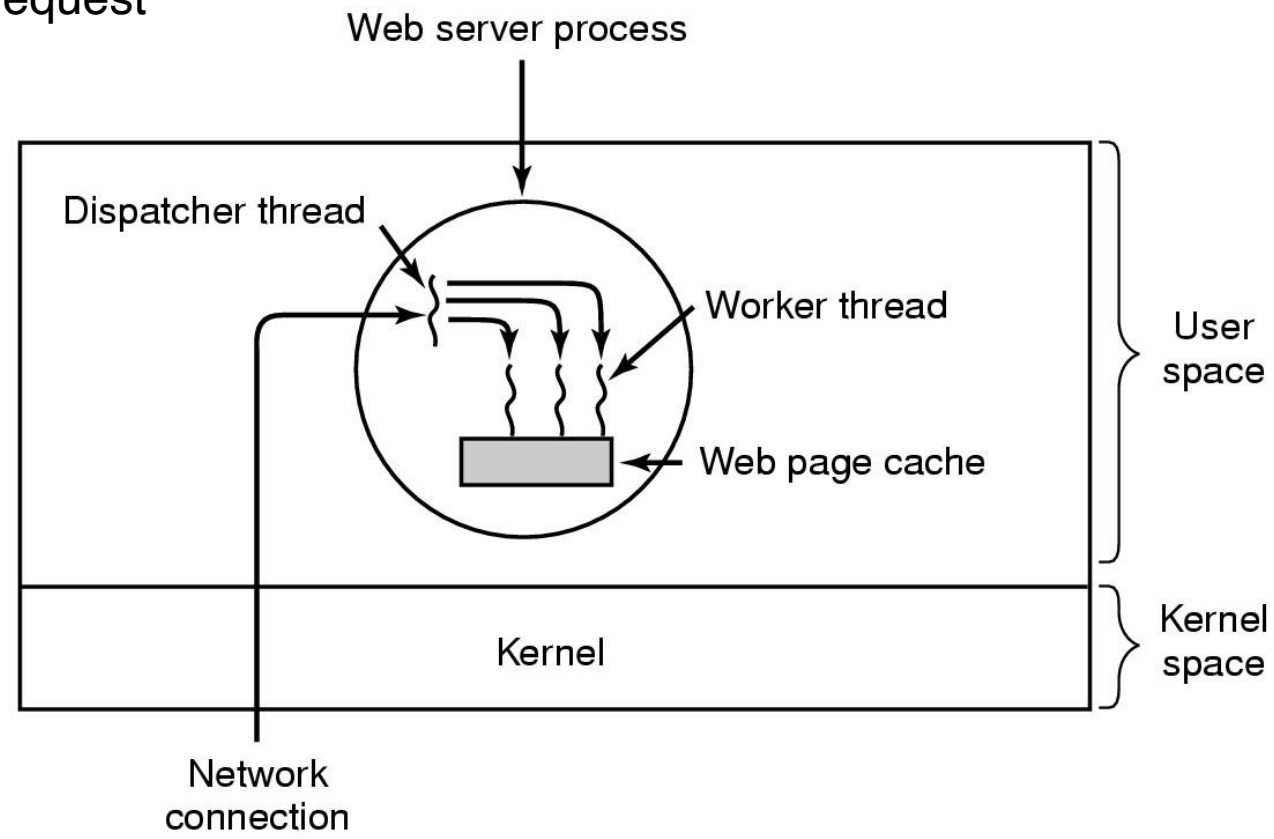
- Overview of Threads
- Processes v.s. Threads
- Pthreads library
- Pthread examples
- Multicore programming models

# Motivation

- Most modern applications are multithreaded
- Threads run within application
- Multiple tasks with the application can be implemented by separate threads
  - Update display
  - Fetch data
  - Spell checking
  - Answer a network request
- **Process creation is heavy-weight while thread creation is light-weight**
- Can simplify code, increase efficiency
- Kernels are generally multithreaded

# Multithreaded Server Architecture

- Web server process
  - Dispatcher thread waits for requests
  - For each request, choose an idle worker thread
  - Worker thread uses blocking system calls to service web request




# Benefits

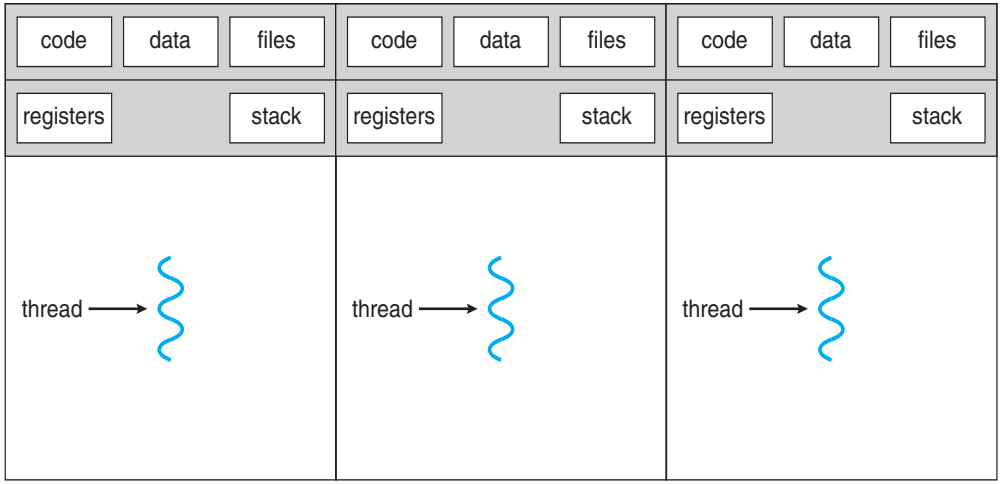
- **Responsiveness –** may allow continued execution if part of process is blocked, especially important for user interfaces
- **Resource Sharing –** threads share resources of process, easier than shared memory or message passing
- **Economy –** cheaper than process creation, thread switching lower overhead than context switching
- **Scalability –** process can take advantage of multiprocessor architectures

# Threads

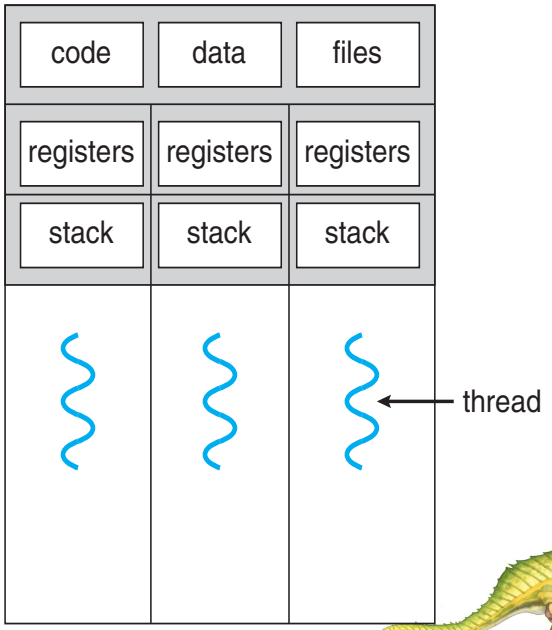
- A Thread is an independent stream of instructions that can be schedule to run as such by the OS.
- Think of a thread as a “procedure” that runs independently from its main program.
- Multi-threaded programs are where several procedures are able to be scheduled to run simultaneously and/or independently by the OS.
- A Thread exists within a process and uses the process resources.
- Threads only duplicate the essential resources it needs to be independently schedulable.
- A thread will die if the parent process dies.
- A thread is “lightweight” because most of the overhead has already been accomplished through the creation of the process.

# Processes v.s. Threads

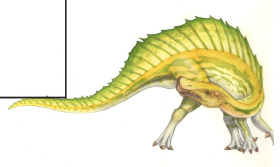
- process:
  - an address space with 1 or more threads executing within that address space, and the required system resources for those threads
- thread:
  - a sequence of control within a process
  - shares the resources in that process
  - Sharing address space requires only one copy of code or data in main memory




Process model



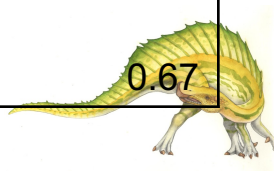

Thread model



# Fork v.s. Threads

- View comparison of forking processes to using a pthreads_create subroutine. Timings reflect 50,000 processes/thread creations.

| PLATFORM | fork() | | pthread_create() | |
|---|---|---|---|---|
| | USER | SYSTEM | USER | SYSTEM |
| AMD 2.4 GHz Opteron (8cpus/node) | 60.08 | 9.01 | 0.19 | 0.43 |
| IBM 1.9 GHz POWER5 p5-575 (8cpus/node) | 30.78 | 27.68 | 0.69 | 1.1 |
| IBM 1.5 GHz POWER4 (8cpus/node) | 48.64 | 47.21 | 1 | 1.52 |
| INTEL 2.4 GHz Xeon (2 cpus/node) | 1.54 | 20.78 | 0.67 | 0.9 |
| INTEL 1.4 GHz Itanium2 (4 cpus/node) | 1.07 | 22.22 | 1.26 | 0.67 |



# Threads Pros and Cons

- Advantages of Threads
  - The overhead for creating a thread is significantly less than that for creating a process
  - Multitasking, i.e., one process serves multiple clients
  - Switching between threads requires the OS to do much less work than switching between processes
- Disadvantages of Threads
  - Writing multithreaded programs require more careful thought
  - More difficult to debug than single threaded programs
    - As long as shared data is not being modified there is no problem
    - But concurrent access to shared data that modify the value of the data can lead to data inconsistency
  - For single processor machines, creating several threads in a program may not necessarily produce an increase in performance

# POSIX Pthreads

- **Thread library** provides programmer with API for creating and managing threads
- Posix defines a thread interface
  - **pthreads**
  - defines how threads should be created, managed, and destroyed
- Unix provides a pthreads library
  - API to create and manage threads
  - you don't need to worry about the implementation details
    - this is a good thing

# Create Threads

- The main() method comprises a single, default thread.
- Define a thread with thread_id
  - pthread_t tid;
  - Pthread_t tid[10];

- pthread_create() creates a new thread and makes it executable.
- The maximum number of threads that may be created by a process in implementation dependent.
- Once created, threads are peers, and may create other threads.

# Create Threads

- Implementation:
  - int pthread_create(pthread_t *tid, const pthread_attr_t *tattr, void*(*start_routine)(void *), void *arg);
    - *tid*: an unsigned long integer that indicates a threads id
    - *tattr*: attributes of the thread – usually NULL
    - *start_routine*: the name of the function the thread starts executing
    - *arg*: the argument to be passed to the start routine – only one
  - after this function gets executed, a new thread has been created and is executing the function indicated by *start_routine*

# Wait for a Thread

- Implementation:
  - int pthread_join(thread_t tid, void **status);
    - *tid*: identification of the thread to wait for
    - *status:* the exit status of the terminating thread – can be NULL
  - the thread that calls this function blocks its own execution until the thread indicated by *tid* terminates its execution
    - finishes the function it started with or
    - issues a *pthread_exit()* command – more on this in a minute

# Example

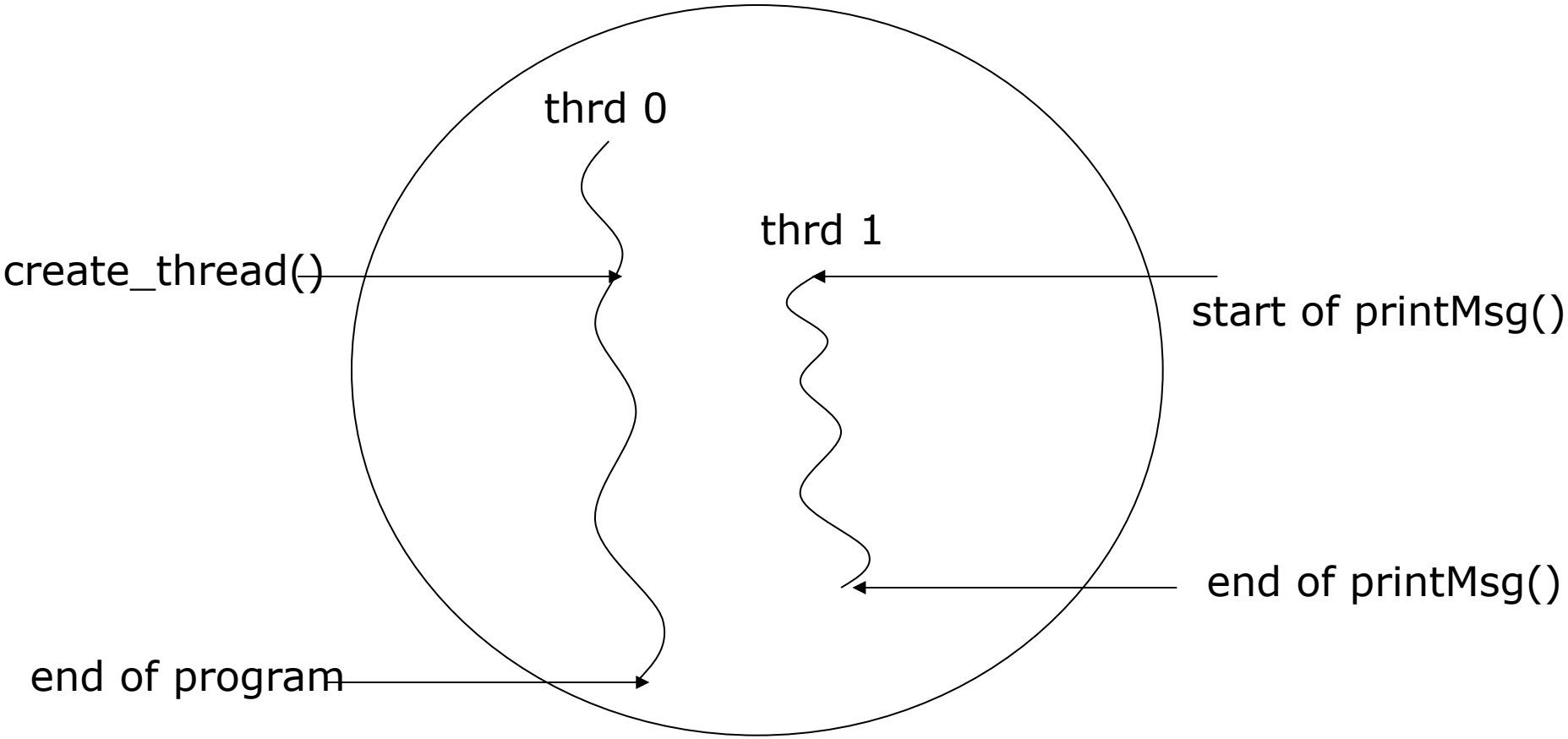
```
#include <stdio.h>
#include <pthread.h>

void *printMsg(void* args) {
    char *msg = (char *)args;
    printf("%s\n", msg);
}

int main(int argc, char** argv) {
    pthread_t tid;
    printf("creating a new thread\n");
    pthread_create(&tid, NULL, printMsg, argv[1]);
    printf("created thread %d\n", tid);
    pthread_join(tid, NULL);
    printf("done\n");

    return 0;
}
```

# Example




**Note:** thrd 0 is the function that contains *main()* – only one *main()* per program

# Exiting a Thread

- pthreads exist in user space and are seen by the kernel as a single process
  - if the *main()* function exits, all of the other threads are terminated
- To have a thread exit, use *pthread_exit()*
- Prototype:
  - void  pthread_exit(void *status);
    - *status:* the exit status of the thread – passed to the *status* variable in the *pthread_join()* function of a thread waiting for this one – NULL usually

# Example Revisited

```
#include <stdio.h>
#include <pthread.h>

void *printMsg(void* args) {
    char *msg = (char *)args;
    printf("%s\n", msg);
    pthread_exit(NULL);
}

int main(int argc, char** argv) {
    pthread_t tid;

    printf("creating a new thread\n");
    pthread_create(&tid, NULL, printMsg, argv[1]);
    printf("created thread %d\n", tid);
    pthread_join(tid, NULL);
    printf("done\n");

    return 0;
}
```

# In-class practice

- Write a program that has two threads. The child thread calculates the sum of 1, 2, 3, …, 100, while the main thread prints out a message “This is the main” after the child thread finishes the sum job.

- Extra problem. Can you modify your program to create two threads, where the first thread calculates the sum from 1 to 50 and the second thread calculates the sum from 51 to 100, and then the main adds these two results and print it out?